

```

package com.pup.taguig.app.controller;

import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicLong;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/activities")
public class ActivityController {

    private final List<Activity> activities = new ArrayList<>();
    private final AtomicLong idCounter = new AtomicLong(1);

    static class Activity {
        private Long id;
        private String name;
        private String description;
        private LocalDateTime createdAt;

        public Activity(Long id, String name, String description,
LocalDateTime createdAt) {
            this.id = id;
            this.name = name;
            this.description = description;
            this.createdAt = createdAt;
        }

        public Long getId() { return id; }
        public void setId(Long id) { this.id = id; }
        public String getName() { return name; }
        public void setName(String name) { this.name = name; }
        public String getDescription() { return description; }
        public void setDescription(String description) { this.description
= description; }
        public LocalDateTime getCreatedAt() { return createdAt; }
        public void setCreatedAt(LocalDateTime createdAt) {
this.createdAt = createdAt; }
    }

    static class ActivityRequest {

```

```

        private String name;
        private String description;

        public String getName() { return name; }
        public void setName(String name) { this.name = name; }
        public String getDescription() { return description; }
        public void setDescription(String description) { this.description
= description; }
    }

    @PostMapping
    public ResponseEntity<?> createActivity(@RequestBody ActivityRequest
request) {
        if (request.getName() == null ||
request.getName().trim().isEmpty()) {
            return ResponseEntity.badRequest().body("Name is required.");
        }
        Activity activity = new Activity(
            idCounter.getAndIncrement(),
            request.getName().trim(),
            request.getDescription(),
            LocalDateTime.now()
        );
        activities.add(activity);
        return ResponseEntity.status(HttpStatus.CREATED).body(activity);
    }

    @GetMapping
    public ResponseEntity<List<Activity>> getAllActivities() {
        return ResponseEntity.ok(activities);
    }

    @GetMapping("/{id}")
    public ResponseEntity<?> getActivityById(@PathVariable Long id) {
        for (Activity activity : activities) {
            if (activity.getId().equals(id)) {
                return ResponseEntity.ok(activity);
            }
        }
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body("Activity with ID " + id + " not found.");
    }

    @PutMapping("/{id}")
    public ResponseEntity<?> updateActivity(@PathVariable Long id,
@RequestBody ActivityRequest request) {
        if (request.getName() == null ||
request.getName().trim().isEmpty()) {
            return ResponseEntity.badRequest().body("Name is required.");
        }
        for (Activity activity : activities) {
            if (activity.getId().equals(id)) {
                activity.setName(request.getName().trim());
                activity.setDescription(request.getDescription());
            }
        }
    }

```

```

        return ResponseEntity.ok(activity);
    }
}
return ResponseEntity.status(HttpStatus.NOT_FOUND)
    .body("Activity with ID " + id + " not found.");
}

@DeleteMapping("/{id}")
public ResponseEntity<?> deleteActivity(@PathVariable Long id) {
    boolean removed = activities.removeIf(activity ->
activity.getId().equals(id));
    if (!removed) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body("Activity with ID " + id + " not found.");
    }
    return ResponseEntity.ok("Activity with ID " + id + " deleted
successfully.");
}
}

```